

OOP

UNIT 8 – POLYMORPHISM AND ABSTRACT CLASSES

Sometimes the safest base class is one that is deliberately incomplete.

FROM VIRTUAL FUNCTIONS TO A STRONGER DESIGN

Unit 7 showed how a virtual function call made through a base reference or pointer can reach the override that belongs to the object's *real type*. The supplement then explained how the `vp`tr and `vtable` make that dynamic binding possible.

Dynamic binding solves an important problem: it selects the correct implementation at run time. However, it does not decide whether the base class should provide a default implementation at all. In some hierarchies, a general base object has no meaningful behavior of its own.

THE PLAY ALL PROBLEM

The music application now holds songs, podcast episodes, and audiobooks. The team wants one Play All operation that can process every item through a common `MediaItem` base class. `MediaItem` therefore declares a virtual play function, and each specialized media class may override it.

On the morning of a demonstration, an Audiobook class is added. Its programmer forgets to override `play`. The code still compiles. During the demonstration, the audiobook reaches the base class's generic playback message instead of audiobook playback. The same design even permits a plain `MediaItem` object that represents no real kind of media at all.

Dynamic binding is working correctly: Audiobook simply has no override, so the inherited generic function runs. The problem is that the class design allowed an incomplete media type to be treated as complete.

LEARNING OBJECTIVES

By the end of this unit, you should be able to:

- Explain why an ordinary virtual function may provide an unsafe default behavior.
- Read and write a pure virtual function declaration using `= 0`.
- Define abstract class and concrete class in plain language.
- Make a derived class concrete by implementing every required operation with override.
- Explain why an abstract class may still have data, constructors, and implemented functions.
- Read UML notation for an abstract class, an abstract operation, and inheritance.
- Use base-class references and pointers to process several concrete types in one loop.
- Define the interface of a class and distinguish a stateful abstract base class from an interface-style class.

THE UNIT 7 SOLUTION—AND ITS LIMIT

The following program uses a normal virtual function. Song replaces the base behavior, but Audiobook silently inherits it. A plain MediaItem is also legal.

```
#include <iostream>
#include <string>
using namespace std;

class MediaItem
{
private:
    string title;

public:
    MediaItem(const string& title) : title(title) { }
    string getTitle() const { return title; }

    virtual void play() const {
        cout << "Generic playback: " << title << endl;
    }

    virtual ~MediaItem() { }
};
```

```

class Song : public MediaItem
{
public:
    Song(const string& title) : MediaItem(title) { }

    void play() const override
    {
        cout << "Playing song: " << getTitle() << endl;
    }
};

class Audiobook : public MediaItem
{
public:
    Audiobook(const string& title) : MediaItem(title) { }
    // The programmer forgot to override play().
};

void playOne(const MediaItem& item)
{
    item.play();
}

```

```
int main()
{
    MediaItem unknown("Untitled");
    Song song("Imagine");
    Audiobook book("The Little Prince");

    playOne(unknown);
    playOne(song);
    playOne(book);
}
```

The program prints:

```
Generic playback: Untitled
Playing song: Imagine
Generic playback: The Little Prince
```

The middle line proves that virtual dispatch works. The first and third lines prove that virtual alone does not express the rule the design needs. A normal virtual function says: a derived class may replace this behavior. It does not say: a derived class must replace this behavior.

THE DESIGN RULE WE NEED

The program needs the compiler to enforce two statements:

1. `MediaItem` is only a *general* category; creating a plain `MediaItem` object must be *impossible*.
2. Every class that can produce objects—`Song`, `PodcastEpisode`, `Audiobook`, and future types—*must* provide its own `play` function.

In plain language, `MediaItem` should describe required behavior without pretending to know how every kind of media plays. C++ expresses that idea with a pure virtual function and an abstract class.

PURE VIRTUAL FUNCTIONS: A REQUIRED OPERATION

A pure virtual function is a virtual function that a concrete derived class is required to implement. Write = 0 after the function declaration:

```
class MediaItem
{
public:
    virtual void play() const = 0;
    virtual ~MediaItem() { }
};
```

- virtual keeps the run-time selection learned in Unit 7.
- void play() const states the exact function signature required from derived classes.
- = 0 marks the function as pure virtual: the base class is not supplying usable default behavior.
- = 0 is special C++ syntax here; it is not an assignment and it does not mean that play returns zero.

The base class now states what every usable media item must be able to do, while each derived class remains responsible for how it does it.

ABSTRACT AND CONCRETE CLASSES

A class is abstract when at least one required pure virtual function is still unimplemented for that class. An abstract class describes a category or an incomplete design, so objects of that exact class cannot be created.

A class is concrete when no pure virtual requirement remains unimplemented. A concrete class is complete enough to create objects.

```
// MediaItem item;          // Error: MediaItem is abstract.

class Audiobook : public MediaItem
{
    // play() is still missing.
};

// Audiobook book;          // Error: Audiobook is also abstract.
```

Declaring Audiobook is legal. C++ permits one incomplete class to derive from another. The compiler rejects the attempt to create an Audiobook object because the required play function is still missing.

REPRESENTING AN ABSTRACT CLASS IN UML

A UML class diagram can show the same design before any C++ code is written. The standard notation writes an abstract class name and each abstract operation in italics. Because italics can be easy to miss, the diagram below also adds {abstract}. This label describes the design; it is not C++ syntax.

- The italic *MediaItem* name and *play(): void* operation, reinforced by {abstract}, identify the abstract class and its required operation.
- + means public and - means private; a solid line with a hollow triangle points to the base class; the non-italic derived class names identify concrete classes.

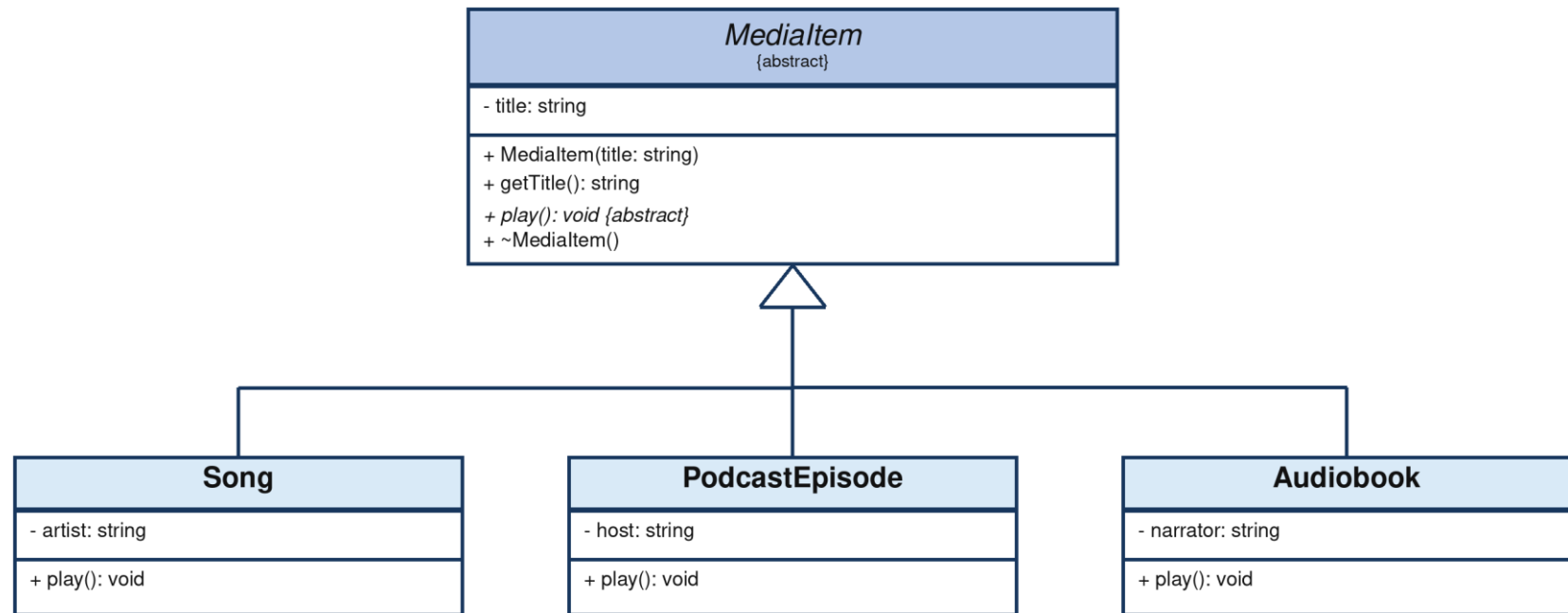


Figure 1. UML representation of the abstract Medialtem hierarchy.

The UML operation `+ play(): void {abstract}` corresponds to the C++ declaration `virtual void play() const = 0;`. UML normally omits language-specific details such as `virtual`, `const`, `override`, and `= 0`, so the diagram and the code express the same rule at different levels.

MAKING A DERIVED CLASS CONCRETE

Song becomes *concrete* by implementing the exact function required by MediaItem. Keep override so the compiler verifies that the signature really matches.

```
class Song : public MediaItem
{
public:
    void play() const override
    {
        cout << "Playing a song" << endl;
    }
};

Song song;                // Legal: Song is concrete.
MediaItem& item = song;    // Legal: a base reference may refer to it.
item.play();              // Dynamic binding selects Song::play().
```

A SIGNATURE MISTAKE KEEPS THE CLASS ABSTRACT

The required signature includes `const`. If the derived function omits it, the function does not override `MediaItem::play() const`. With `override`, the compiler reports the mistake at the function declaration.

```
class BrokenSong : public MediaItem
{
public:
    void play() override { } // Error: does not override play() const.
};
```

Removing `override` is not a fix. It merely hides the useful diagnostic, and `BrokenSong` still has not implemented the required `const` function.

ABSTRACT DOES NOT MEAN EMPTY

An abstract class cannot produce objects of its own exact type, but its base part still exists inside every derived object. It may therefore store genuinely shared data and implement genuinely shared behavior.

- Data members are allowed.
- Constructors are allowed and initialize the base part of derived objects.
- Ordinary implemented member functions are allowed.
- Some virtual functions may have implementations while other functions are pure virtual.
- Only direct objects of the abstract class itself are forbidden.

```
class MediaItem
{
private:
    string title;

public:
    MediaItem(const string& title) : title(title) { }
    string getTitle() const { return title; }
    void printTitle() const { cout << title << endl; }
```

```
virtual void play() const = 0;  
virtual ~MediaItem() { }  
};
```

When a Song is constructed, MediaItem's constructor still runs first and initializes the title. Abstract changes whether MediaItem can stand alone; it does not erase the MediaItem part of a derived object.

PAUSE AND PREDICT

Return to the small MediaItem and Song definitions above. For each line, decide whether it compiles and explain why:

```
MediaItem first;  
Song second;  
MediaItem* third = &second;  
third->play();
```

1. Which class is abstract? Which class is concrete?
2. Why may third point to second even though a MediaItem object cannot be created?
3. Which play function runs, and which Unit 7 mechanism selects it?

CASE STUDY: THE CORRECTED MUSICLIBRARY

The complete program now uses `MediaItem` as an abstract base class. It holds the shared title, implements `getTitle`, and requires `play`. Each derived class supplies its own playback behavior. `MusicLibrary` stores non-owning base pointers so one collection can refer to objects of several concrete types.

```
#include <iostream>
#include <string>
using namespace std;

class MediaItem
{
private:
    string title;

public:
    MediaItem(const string& title) : title(title) { }

    string getTitle() const { return title; }

    virtual void play() const = 0;
    virtual ~MediaItem() { }
};
```



```

class Song : public MediaItem
{
private:
    string artist;

public:
    Song(const string& title, const string& artist)
        : MediaItem(title), artist(artist) { }

    void play() const override
    {
        cout << "Playing song: " << getTitle()
              << " by " << artist << endl;
    }
};

class PodcastEpisode : public MediaItem
{
private:
    string host;

public:

```

```

    PodcastEpisode(const string& title, const string& host)
        : MediaItem(title), host(host) { }

    void play() const override
    {
        cout << "Playing podcast: " << getTitle()
              << " with " << host << endl;
    }
};

class Audiobook : public MediaItem
{
private:
    string narrator;

public:
    Audiobook(const string& title, const string& narrator)
        : MediaItem(title), narrator(narrator) { }

    void play() const override
    {
        cout << "Playing audiobook: " << getTitle()
              << " narrated by " << narrator << endl;
    }
};

```

```

    }
};

class MusicLibrary
{
private:
    static const int MAX_ITEMS = 10;
    MediaItem* items[MAX_ITEMS];
    int count;

public:
    MusicLibrary() : count(0) { }

    bool add(MediaItem& item)
    {
        if (count == MAX_ITEMS)
            return false;

        items[count++] = &item;
        return true;
    }

    void playAll() const

```

```
    {  
        for (int i = 0; i < count; ++i)  
            items[i]->play();  
    }  
};  
  
int main()  
{  
    Song song("Imagine", "John Lennon");  
    PodcastEpisode episode("Deep Focus", "Ana Reyes");  
    Audiobook book("The Little Prince", "David Suchet");  
  
    MusicLibrary library;  
    library.add(song);  
    library.add(episode);  
    library.add(book);  
  
    library.playAll();  
}
```

The program prints:

```
Playing song: Imagine by John Lennon  
Playing podcast: Deep Focus with Ana Reyes  
Playing audiobook: The Little Prince narrated by David Suchet
```

READING THE CASE STUDY IN FIVE STEPS

1. `MediaItem` centralizes the data and behavior that are truly shared: the title and its accessor.
2. `MediaItem::play()` is pure virtual because there is no honest generic playback algorithm.
3. `Song`, `PodcastEpisode`, and `Audiobook` each implement `play()` const and become concrete.
4. `MusicLibrary` stores `MediaItem*` values, so it preserves each complete derived object instead of slicing it.
5. `playAll` uses the same call expression for every item while dynamic binding selects different overrides.

TRACING PLAYALL BACK TO THE SUPPLEMENT

Consider the iteration in which `items[i]` points to the Audiobook object:

1. The declared pointer type is `MediaItem*`.
2. The real object type is `Audiobook`.
3. `play` is virtual, so the call uses the object's `vptr` and the `Audiobook` `vtable`.
4. The `play` slot for `Audiobook` identifies `Audiobook::play()`.
5. The same loop works for the other objects because only the real object—and therefore the selected `vtable` entry—changes.

The new feature here is not a different dispatch mechanism. Pure virtual `play` uses the same dynamic binding learned in the supplement. The new feature is the compile-time rule that prevents an instantiable class from leaving the `play` slot without an appropriate final override.

REFERENCES, POINTERS, AND LIFETIME

An abstract type may still be used as a reference or pointer type. Those variables do not contain an abstract object; they refer to a complete concrete object.

- Use a base reference for a function that needs one existing object and does not store it.
- Use a base pointer when a collection stores addresses or when the relationship may be reseated.
- Do not pass polymorphic objects by value; Unit 7 showed that value copying slices the derived part.
- MusicLibrary's pointers are non-owning: main creates the objects and keeps them alive while the library uses them.

VIRTUAL DESTRUCTOR: A RULE CARRIED FORWARD

MediaItem still needs a virtual destructor because it is a polymorphic base class. If a later design allocates a Song dynamically and deletes it through MediaItem*, the derived destructor must run first. Being abstract does not make the destructor virtual automatically; the class must declare it virtual.

THE INTERFACE OF A CLASS

The interface of a class is the set of public operations through which other code can use its objects. It answers the question What can code ask an object to do? The private data and the code inside the functions are implementation details: they answer How does the object do it?

For `MediaItem`, `play` and `getTitle` belong to the public interface. The title, artist, host, narrator, and function bodies belong to the implementation of particular classes.

A pure virtual function lets a base class place an operation in the shared public interface without providing the one behavior that every derived class must use. The base specifies what is required; the derived class supplies how.

INTERFACE-STYLE CLASSES IN C++

C++ has no interface keyword. The phrase interface-style class describes a design convention: an abstract class whose main purpose is to name a capability. It usually stores no object state and declares only pure virtual operations plus a virtual destructor.

```
class Playable
{
public:
    virtual void play() const = 0;
    virtual void stop() = 0;
    virtual ~Playable() { }
};

class Doorbell : public Playable
{
public:
    void play() const override { /* ring */ }
    void stop() override { /* silence */ }
};
```

Playable says only that a usable derived object can play and stop. It does not store a title, duration, volume, or device type. Doorbell provides the behavior and is therefore concrete.

When Playable is modeled explicitly as a UML interface, write the stereotype «interface» above its name. A concrete class such as Doorbell is then connected to it by a dashed realization line ending in a hollow triangle. By contrast, the MediaItem diagram uses solid generalization lines because MediaItem is modeled as an abstract base class with shared state and implementation.

TWO RELATED DESIGNS, NOT TWO C++ KEYWORDS

MediaItem and Playable are both abstract classes in the C++ language. The difference is the design role they play:

- Stateful abstract base class: MediaItem represents one related family, stores shared state, implements shared behavior, and also requires specialized behavior.
- Interface-style class: Playable mainly names a capability, normally stores no state, and leaves the required operations to derived classes.

Use a stateful abstract base when the derived types genuinely share identity, data, and implementation. Use an interface-style class when the important idea is only a capability that different concrete classes can promise.

GUIDED PRACTICE: ADD RADIOSTREAM

Add a fourth concrete type to the MusicLibrary without changing MediaItem or the library loop. RadioStream stores a station name and implements play() const.

```
class RadioStream : public MediaItem
{
private:
    string station;

public:
    RadioStream(const string& title, const string& station)
        : MediaItem(title), station(station) { }

    // Write the required override here.
};
```

1. Write the exact play() const override and include the station in its output.
2. Create one RadioStream in main and add it to the library.
3. Predict which existing lines of MusicLibrary must change. Then verify that the answer is none.
4. Explain which compiler error protects the design if the override is omitted.

INDEPENDENT CHECK: DESIGN BEFORE CODING

A delivery application handles `BicycleCourier`, `CarCourier`, and `DroneCourier`. Every usable courier must calculate an estimated arrival time, but all couriers share an identifier and a current location.

1. Choose a name for the abstract base class.
2. List the shared data and one implemented shared operation.
3. Write the declaration of the required pure virtual `estimateTravelTime` function.
4. Show one concrete derived class with a matching override.
5. Explain why this base is more like `MediaItem` than the state-free `Playable` class.

DESIGN GUIDELINES

- Make a base class abstract when the general category has no meaningful standalone object.
- Use a pure virtual function when every concrete derived class must provide the operation honestly.
- Do not invent a misleading default implementation merely to make the base class instantiable.
- Keep required signatures small, stable, and marked const when the operation should not modify the object.
- Mark every intended derived implementation with override.
- Keep shared state private in a stateful abstract base and expose it through protected or public functions as appropriate.
- Use references and pointers for polymorphic objects, state pointer ownership, and keep a virtual base destructor.
- Use the phrase interface-style class only after deciding that the class represents a capability rather than shared state and implementation.

COMMON BEGINNER MISTAKES

- Thinking abstract means that the class cannot have a constructor or data members.
- Thinking `= 0` assigns zero to a function or makes the function return zero.
- Trying to create an object of an abstract class.
- Forgetting one inherited pure virtual function and leaving the derived class abstract.
- Changing `const` or a parameter and therefore failing to override the required function.
- Removing override to silence a compiler error instead of correcting the signature.
- Thinking an abstract class cannot be used as a pointer or reference type.
- Assuming the word `interface` names a separate C++ keyword or language construct.

SUMMARY

Unit 7's ordinary virtual function allows derived classes to replace a base implementation. A pure virtual function strengthens that design: every concrete derived class must provide the required operation. The `= 0` syntax marks that requirement.

A class with any unimplemented pure virtual requirement is abstract and cannot be instantiated. A class that implements all such requirements is concrete. Abstract classes may still store shared data, run constructors, and implement common behavior because their base parts exist inside concrete derived objects.

In UML, italics or `{abstract}` identify abstract classes and operations. A solid line with a hollow triangle represents inheritance and points toward the base class.

An interface is the public set of operations through which code uses an object. In C++, an interface-style class is simply an abstract class used mainly to name a capability, usually without state. It is a design style, not a separate keyword.

REVIEW QUESTIONS

1. What two design errors were still possible when `MediaItem::play` had a normal virtual implementation?
2. What stronger rule does a pure virtual function express?
3. What does `= 0` mean in a virtual function declaration?
4. Define abstract class and concrete class in your own words.
5. Why is an abstract class allowed to have a constructor?
6. What happens when a derived class does not implement every inherited pure virtual function?
7. How are an abstract class, an abstract operation, and inheritance shown in UML?
8. Why can a `MediaItem*` point to a `Song` even though a `MediaItem` object cannot be created?
9. How does `override` help when `const` is accidentally omitted?
10. Which `MediaItem` behavior is shared, and which behavior is dynamically selected?
11. How does the supplement's vtable explanation apply to `items[i]->play()`?
12. What does non-owning mean for the pointers stored by `MusicLibrary`?

13. Why does an abstract polymorphic base class still need a virtual destructor?
14. What is the interface of a class?
15. How does a stateful abstract base class differ from an interface-style class?
16. When `RadioStream` is added, why does the existing `playAll` loop remain unchanged?

EXIT CHECK

Complete these three sentences without looking back:

1. A normal virtual function allows a derived class to ..., while a pure virtual function requires it to
2. A class is abstract when
3. In C++, an interface-style class is

LOOKING AHEAD

The `MusicLibrary` now rejects incomplete media types at compile time and processes new concrete types through one stable public interface. The next unit examines how C++ programs detect, report, and handle exceptional situations safely.